Digital Nurture 5.0

Python Full Stack Engineer Track

HANDS-ON EXERCISE BOOK

QA Concepts & Test Automation — Selenium Basics

7 Hands-On Exercises

How to Use This Exercise Book

This exercise book covers QA Concepts & Test Automation for the Digital Nurture 5.0 Deep Skilling Program. The 7 hands-on exercises are organised to build your QA knowledge systematically — beginning with QA concepts and the testing lifecycle, moving through test automation theory, and culminating in practical Selenium WebDriver automation with the Page Object Model.

All practical Selenium exercises use a consistent test target — a publicly available practice web application — so your automation scripts build progressively from simple interactions to a full POM-based test suite.

Software & Tools Required

- Python 3.10+ — https://www.python.org/downloads/
- VS Code — https://code.visualstudio.com/download
- Google Chrome browser (latest) — https://www.google.com/chrome/
- ChromeDriver (matching your Chrome version) — https://chromedriver.chromium.org/downloads
- pip packages: selenium, pytest, webdriver-manager
- Selenium IDE (browser extension) — https://www.selenium.dev/selenium-ide/
- Practice test site: https://www.lambdatest.com/selenium-playground/

Submission Guidelines

- Organise solutions in a folder named SeleniumBasics/<YourName>/
- Hands-On 1–3 are written exercises — submit as .md or .txt files inside written_exercises/
- Hands-On 4–7 are coding exercises — submit .py files inside automation_scripts/
- Include a requirements.txt: selenium, pytest, webdriver-manager
- Push to your GitHub repository and share the URL with your POC

NOTE	Hands-On 1–3 test your understanding of QA theory. Hands-On 4–7 are hands-on coding with Selenium WebDriver and Python.

Difficulty Guide

Level	Hands-On	What to Expect
Beginner	1, 2	QA concepts, defect lifecycle, SDLC vs TDLC
Intermediate	3, 4, 5	Test automation theory, Selenium setup, locators
Advanced	6, 7	Selenium waits, pytest integration, Page Object Model

Practice Application & Test Scenario

All Selenium automation exercises target the LambdaTest Selenium Playground — a publicly available web app designed specifically for automation practice. It contains forms, buttons, dropdowns, alerts, and tables that cover all major Selenium interaction patterns.

> URL: https://www.lambdatest.com/selenium-playground/ — This site is always available, requires no login, and includes components specifically designed to practise all Selenium locator strategies and interaction commands.

Test Scenarios to Automate (Hands-On 4–7)

#	Scenario	What to Test
1	**Simple Form Demo**	Input text, click Submit, assert message appears
2	**Checkbox Demo**	Check/uncheck checkboxes, verify state
3	**Select Dropdown**	Select options, verify selected value
4	**Bootstrap Alerts**	Click button to show alert, assert alert text
5	**Input Form Submit**	Fill complete form, submit, assert success
6	**Table Sort**	Click column header, verify table sort order

HANDS-ON 1 [Beginner]

QA Concepts, Functional Testing & Defect Lifecycle

Topics Covered:

✓ **QA Roles & Responsibilities**	✓ **Types of Testing — Functional vs Non-Functional**
✓ **Test Levels — Unit, Integration, System, UAT**	✓ **Black-Box vs White-Box Testing**
✓ **Defect Lifecycle & Management**	✓ **Test Case Writing**

This hands-on establishes the theoretical foundation for everything that follows. Strong QA fundamentals — knowing when to test, what to test, and how to report defects — are as important as automation skills.

FORMAT	Submit your answers as a written document (qa_concepts.md). Use clear headings for each task.

Task 1: Map Testing Types to a Real System

Goal: Classify and apply different testing types to the Course Management API built in earlier hands-on exercises.

Steps:

1. For the Course Management API, identify and describe one concrete test case for each of the following testing types: Unit Testing (test a single function in isolation), Integration Testing (test two components working together, e.g., the API endpoint + database), System Testing (test a full end-to-end flow from API request to database response), User Acceptance Testing (test from the perspective of an actual college admin user).
2. Classify each test case as either Functional (does it do what it should?) or Non-Functional (how well does it do it?). Give one non-functional test example for the API (e.g., performance, security, or reliability).
3. Explain the difference between Black-Box Testing (testing without knowledge of internal code) and White-Box Testing (testing with knowledge of the code). State which type a QA tester typically performs and which a developer performs.
4. Write 3 formal test cases for the POST /api/courses/ endpoint in a table format with columns: Test Case ID, Description, Preconditions, Test Steps, Expected Result, Actual Result (leave blank), Pass/Fail (leave blank).

Hint:

- A test case is not a test script — it is a documented procedure that anyone can follow and get the same result.
- Non-functional tests answer 'how well' questions: How fast? How secure? How reliably? They are equally important as functional tests.

Expected Outcome: Written document contains 4 typed test cases (one per test level), 1 non-functional example, a clear black-box vs white-box explanation, and 3 formal test cases in table format.

Task 2: Defect Lifecycle & Severity Classification

Goal: Understand how defects are reported, tracked, and resolved in a professional QA process.

Steps:

5. Draw or describe (in text) the complete defect lifecycle with all states: New → Assigned → Open → Fixed → Retest → Verified → Closed. Also describe the Rejected and Deferred paths.
6. For each of the following hypothetical bugs in the Course Management API, classify the Severity (Critical / High / Medium / Low) and Priority (P1 / P2 / P3 / P4), and justify your classification: a)

POST /api/courses/ returns 500 Internal Server Error for all requests. b) Course names longer than 150 characters are silently truncated without an error. c) The /docs Swagger page has a typo in the API description. d) Login with correct credentials occasionally returns 401 on the first attempt (intermittent).

7. Write a complete defect report for bug (a) above using standard fields: Defect ID, Title, Environment, Build Version, Severity, Priority, Steps to Reproduce, Expected Result, Actual Result, Attachments (state 'screenshot of 500 error').
8. Explain the difference between Severity and Priority with a real-world example where High Severity does not mean High Priority.

Hint:

- Severity measures the impact of the defect on the system. Priority measures how urgently it needs to be fixed — a cosmetic bug on the CEO's dashboard might have Low Severity but High Priority.
- Intermittent bugs (bug d) are typically marked High Priority despite potentially lower severity — they are hard to reproduce and indicate deeper instability.

Expected Outcome: Document contains the full defect lifecycle diagram, severity/priority classifications with justifications, a complete defect report, and a clear severity vs priority explanation.

HANDS-ON 2 [Beginner]

SDLC vs TDLC — V-Model & Agile QA Integration

Topics Covered:

✓ **SDLC Phases**	✓ **TDLC Phases**
✓ **V-Model Mapping**	✓ **Entry & Exit Criteria**
✓ **Agile QA Integration**	✓ **Shift-Left Testing Principle**

Understanding how software development and testing lifecycles relate to each other is fundamental to being an effective QA engineer. In this hands-on, you will map SDLC to TDLC using the V-Model, define entry/exit criteria, and understand how QA works in an Agile team.

FORMAT	Submit as v_model_analysis.md. Diagrams can be described in text or drawn as ASCII art.

Task 1: V-Model Mapping

Goal: Create a complete V-Model diagram mapping each SDLC phase to its corresponding TDLC phase.

Steps:

9. Draw or describe the V-Model showing the left side (development phases: Requirements → System Design → Architecture Design → Module Design → Coding) and the right side (testing phases: Acceptance Testing → System Testing → Integration Testing → Unit Testing) with Coding at the bottom vertex.
10. For each pair of SDLC ↔ TDLC phases, describe what test artifact is produced during the development phase. Example: Requirements phase → Acceptance Test plan is prepared during this phase.
11. For each TDLC phase, define: Entry Criteria (what must be true before this testing phase begins) and Exit Criteria (what must be true before this phase is considered complete). Document these for all four testing levels.
12. Identify two specific places in the V-Model where QA should engage during the Course Management API project — not just during testing phases.

Hint:

- QA engagement during requirements review (left side of V-Model) catches ambiguities before any code is written — this is far cheaper than finding the same issue during system testing.
- Exit criteria typically include: all planned test cases executed, defect count below a threshold, no open critical/high defects.

Expected Outcome: Document shows the complete V-Model with all phase mappings, test artifacts per phase, entry/exit criteria for all 4 testing levels, and 2 early QA engagement points.

Task 2: Agile QA and Shift-Left Testing

Goal: Understand how QA integrates into Agile sprints and the Shift-Left principle.

Steps:

13. In a traditional Waterfall project, testing happens after development is complete. Describe 3 problems this causes for the Course Management API project.
14. In Agile, QA is involved from Sprint Planning. Describe what a QA engineer does in each Agile ceremony: Sprint Planning (defining acceptance criteria), Daily Standup (blocking issues), Sprint Review (demo testing), Retrospective (process improvement).
15. The Shift-Left principle means testing activities move earlier in the SDLC. List 4 concrete Shift-Left practices and describe how each would apply to the Course Management API: (a) reviewing requirements for testability, (b) writing test cases before code (TDD/BDD), (c) static code analysis, (d) API contract testing before integration.

16. Write the Acceptance Criteria for the following user story in Given-When-Then (Gherkin) format: 'As a college admin, I want to create a new course, so that students can enroll in it.' Cover: happy path, duplicate course code, and missing required fields.

Hint:

- Acceptance Criteria written in Given-When-Then format are directly executable as automated tests using tools like Behave (Python) or Cucumber (Java/JS).
- Shift-Left does not mean developers test everything — it means QA and developers collaborate earlier, preventing defects rather than just finding them.

Expected Outcome: Document describes 3 waterfall problems, QA role in all 4 Agile ceremonies, 4 Shift-Left practices applied to the API, and 3 Given-When-Then acceptance criteria scenarios.

HANDS-ON 3 [Intermediate]

Test Automation Process, Lifecycle & Framework Types

Topics Covered:

✓ **When to Automate — Decision Criteria**	✓ **Automation Test Lifecycle**
✓ **Selecting Test Cases for Automation**	✓ **Framework Types — Linear, Modular, Data-Driven, Keyword-Driven, Hybrid**
✓ **Automation ROI**	

Not every test should be automated. In this hands-on, you will learn how to decide what to automate, understand the automation lifecycle, and compare the five main automation framework types before writing a single line of Selenium code.

FORMAT	Submit as automation_strategy.md. This is a planning document — think like a QA lead, not a developer.

Task 1: Automation Decision and Test Case Selection

Goal: Apply the criteria for deciding what to automate and select the right candidates.

Steps:

17. List and explain 5 criteria for deciding whether a test case should be automated. Apply each criterion to this scenario: 'Test that the POST /api/courses/ endpoint returns 201 with the correct course data when valid input is provided.'
18. From the following list of test cases for the Course Management API, mark each as 'Automate' or 'Manual' and justify your decision: (a) Regression test for all CRUD endpoints after every code change. (b) Exploratory testing of a new search feature. (c) Performance test: 100 concurrent users calling GET /api/courses/. (d) UI test for the login form. (e) Verify the API documentation (Swagger) is accurate. (f) Smoke test: verify the API is reachable after deployment.
19. Define the term 'test automation ROI'. Given that automating one regression test takes 4 hours and running it manually takes 30 minutes, calculate how many runs are needed before the automation pays for itself. Account for a 20% maintenance overhead per run after the 10th run.
20. Describe what a 'flaky test' is, give one example, and list 3 strategies to prevent or fix flaky tests in a Selenium suite.

Hint:

- Good candidates for automation: repetitive, high-risk, regression, data-driven. Poor candidates: exploratory, UI-heavy, one-time, or rapidly-changing tests.
- Flaky tests undermine confidence in the entire test suite — a test that sometimes passes and sometimes fails is often worse than no test at all.

Expected Outcome: Document covers 5 automation criteria with justifications, 6 manual/automate decisions with reasons, an ROI calculation, and a flaky test analysis.

Task 2: Compare Automation Framework Types

Goal: Understand and compare the five automation framework architectures.

Steps:

21. For each of the 5 framework types (Linear, Modular, Data-Driven, Keyword-Driven, Hybrid), provide: a one-paragraph description, one advantage, one disadvantage, and a brief example of when you would use it for the Course Management system.
22. The team is building a Selenium suite for the Course Management frontend. They need to: test login with 50 different user/password combinations, reuse login steps across 20 test cases, and support both technical and non-technical team members writing tests. Which framework type (or combination) would you recommend? Justify your answer.

23. Draw or describe the folder structure you would create for a Hybrid framework for the Course Management frontend tests. Include: test data files, page object files, utility files, test files, and configuration.

Hint:

- The Hybrid framework is the most common in real projects — it combines the reusability of Modular, the parameterisation of Data-Driven, and optionally the abstraction of Keyword-Driven.
- There is no universally 'best' framework — the choice depends on team skill, project size, and how frequently requirements change.

Expected Outcome: Document contains structured comparisons of all 5 framework types, a justified recommendation for the scenario, and a clearly described hybrid folder structure.

HANDS-ON 4 [Intermediate]

Selenium WebDriver Setup, Browser Drivers & Basic Commands

Topics Covered:

✓ **Selenium Architecture — WebDriver, Grid, IDE**	✓ **Browser Driver Setup (ChromeDriver)**
✓ **webdriver-manager for Auto Driver Management**	✓ **WebDriver Commands — navigate, get, close, quit**
✓ **Window & Frame Handling**	✓ **Taking Screenshots**

This is where the automation coding begins. You will set up Selenium WebDriver with Python, understand the Selenium architecture, and write your first browser automation scripts against the LambdaTest Selenium Playground.

INSTALL	pip install selenium webdriver-manager — webdriver-manager auto-downloads the correct ChromeDriver, so you do not need to manage driver versions manually.

Task 1: Selenium Architecture and Environment Setup

Goal: Understand the Selenium component architecture and set up your automation environment.

Steps:

24. In a comment block at the top of setup_test.py, describe the three main Selenium components: WebDriver (what it is, how it communicates with the browser), Selenium Grid (what problem it solves — parallel execution on multiple machines/browsers), and Selenium IDE (what it is used for — record and playback, code generation).
25. Install Selenium and webdriver-manager. Write a minimal script that: imports Chrome WebDriver using webdriver-manager (from webdriver_manager.chrome import ChromeDriverManager), opens Chrome, navigates to https://www.lambdatest.com/selenium-playground/, prints the page title, and closes the browser.
26. Add implicit wait: driver.implicitly_wait(10). Explain in a comment why setting implicit wait globally is considered a bad practice compared to explicit waits (covered in Hands-On 5).
27. Modify the script to run in headless mode using ChromeOptions: options.add_argument('--headless'). Verify the title is still printed correctly without a visible browser window.

Expected Outcome:

Task 2: WebDriver Navigation and Window Commands

Goal: Use WebDriver commands to navigate, handle multiple windows, and capture screenshots.

Steps:

28. Write a script that: opens the Selenium Playground, navigates to the Simple Form Demo page (click the link), asserts the URL contains 'simple-form-demo' using assert 'simple-form-demo' in driver.current_url, then navigates back using driver.back().
29. Open a new browser tab using driver.execute_script('window.open("https://www.google.com");'). Use driver.window_handles to list all open tabs. Switch to the new tab using driver.switch_to.window(driver.window_handles[1]). Print the title of the Google tab.
30. Switch back to the original tab (window_handles[0]) and take a screenshot: driver.save_screenshot('playground_screenshot.png'). Verify the file is created.
31. Demonstrate get_window_size() and set_window_size(1280, 800) — document in a comment why consistent window size matters for responsive UI automation.

Expected Outcome: Script navigates, asserts URL, opens a second tab, prints Google title, switches back, and saves a screenshot file.

HANDS-ON 5 [Intermediate]

Locators — ID, Name, XPath, CSS Selectors & Explicit Waits

Topics Covered:

✓ **Locator Strategies — ID, Name, Class, Tag, XPath, CSS**	✓ **Writing Robust XPath Expressions**
✓ **CSS Selector Patterns**	✓ **WebDriverWait & Expected Conditions**
✓ **Implicit vs Explicit vs Fluent Waits**	✓ **Avoiding Hard-Coded Sleeps**

Choosing the right locator strategy and handling dynamic elements with proper waits are the two most critical skills for writing reliable Selenium automation. Fragile locators and sleep() calls are the leading causes of flaky test suites.

TARGET	All tasks in this hands-on use the LambdaTest Selenium Playground at https://www.lambdatest.com/selenium-playground/

Task 1: Locator Strategies — From Simple to Robust

Goal: Practise all six locator strategies and understand when to use each one.

Steps:

32. Open the Simple Form Demo page. Use browser DevTools (F12 → Elements) to inspect the message input field. Locate it using each of the following strategies and verify each works in your script: By.ID, By.NAME, By.CLASS_NAME, By.TAG_NAME (locate the input tag), By.XPATH with absolute path, By.XPATH with relative path using attributes.
33. Now locate the same element using By.CSS_SELECTOR. Write 3 different CSS selectors for the same element: by ID (#id), by attribute ([name='value']), and by parent-child relationship (div > input).
34. Open the Checkbox Demo page. Use XPath with text(): //label[text()='Option 1'] to find the first checkbox label. Use contains(): //label[contains(text(),'Option')] to find all option labels.
35. Rank the 6 locator strategies from most to least preferred for maintainable automation. Justify your ranking in comments — consider: uniqueness, brittleness to HTML changes, and readability.

Hint:

- ID is the most preferred locator — unique, fast, and readable. XPath with absolute paths (/html/body/div[2]/...) is the worst — it breaks with any HTML structure change.
- CSS selectors are generally faster than XPath and sufficient for most locating needs. Use XPath only when CSS cannot express the condition (e.g., text-based or parent-axis traversal).

Expected Outcome: All 6 locator strategies successfully find the element. 3 CSS selectors work. XPath text() and contains() locate the checkbox labels.

Task 2: WebDriverWait and Expected Conditions

Goal: Replace all time.sleep() calls with explicit waits using WebDriverWait and ExpectedConditions.

Steps:

36. Open the Bootstrap Alerts demo. Write a script that: clicks the 'Success Message' button, then waits for the success alert div to become visible using WebDriverWait(driver, 10).until(EC.visibility_of_element_located((By.CSS_SELECTOR, '.alert-success'))). Assert the alert text contains 'successfully'.
37. Demonstrate why time.sleep(3) is bad: write the same test with time.sleep(3) and time it. Then write it with explicit wait and time it. Comment on the difference — the explicit wait version should be faster on fast machines and more reliable on slow ones.
38. Add a wait for an element to be clickable before clicking it: EC.element_to_be_clickable(). Explain in a comment the difference between visibility_of_element_located (element is visible in DOM) and element_to_be_clickable (visible AND enabled AND not obscured).

39. Use FluentWait to poll every 500ms with a maximum of 10 seconds and ignore NoSuchElementException during polling. Apply it to wait for a dynamically-loaded table row.

Expected Outcome:

HANDS-ON 6 [Advanced]

Running Selenium Tests with pytest — Fixtures, Assertions & Reporting

Topics Covered:

✓ **pytest Basics — Test Discovery, Fixtures**	✓ **conftest.py — Shared Fixtures**
✓ **pytest Assertions**	✓ **Parameterised Tests**
✓ **HTML Test Reports**	✓ **Screenshot on Failure**

Individual Selenium scripts are not a test suite. In this hands-on, you will integrate your scripts into pytest to get proper test discovery, fixtures for browser setup/teardown, parameterisation, and HTML reports.

INSTALL	pip install pytest pytest-html — pytest-html generates a visual HTML report of test results

Task 1: Organise Scripts into pytest Tests

Goal: Restructure automation scripts as pytest test functions with proper setup and teardown.

Steps:

40. Create a test file test_playground.py. Rename your script functions to start with test_ so pytest discovers them: def test_simple_form_submission():, def test_checkbox_interaction():.
41. Create conftest.py in the same folder. Define a driver fixture with function scope: @pytest.fixture(scope='function'). Inside, initialise the Chrome WebDriver, yield the driver to the test, then call driver.quit() in the teardown (after yield). All tests should receive the driver via parameter injection, not create their own.
42. Write test_simple_form_submission(driver): open the Simple Form Demo, enter 'Hello Selenium' in the message input, click Submit, wait for the message display element, and assert its text equals 'Hello Selenium'.
43. Write test_checkbox_demo(driver): open the Checkbox Demo, click the first checkbox, assert it is selected (is_selected()), click it again, assert it is deselected.
44. Run pytest test_playground.py -v and verify both tests pass with the driver properly set up and torn down.

Hint:

- scope='function' means a new browser instance per test — tests are fully isolated. scope='session' reuses one browser for all tests — faster but tests can interfere with each other.
- The yield in a fixture splits it into setup (before yield) and teardown (after yield) — equivalent to setUp/tearDown in unittest.

Expected Outcome: pytest -v shows test_simple_form_submission PASSED and test_checkbox_demo PASSED. Each test gets a fresh browser instance.

Task 2: Parameterisation, Reporting and Screenshot on Failure

Goal: Use pytest's advanced features to increase test coverage and improve reporting.

Steps:

45. Parameterise the form submission test with 3 different input values using @pytest.mark.parametrize('message', ['Hello', 'Selenium Automation', '12345']). All 3 parameter sets should generate separate test runs.
46. Add a conftest.py hook to capture a screenshot on test failure: implement pytest_runtest_makereport and use request.node to check if the test failed, then call driver.save_screenshot(f'{test_name}_failure.png').

47. Generate an HTML report: pytest test_playground.py --html=report.html --self-contained-html. Open report.html and verify it shows test names, pass/fail status, and duration.
48. Add a session-scoped fixture for a base_url constant = 'https://www.lambdatest.com/selenium-playground/'. Use it in all tests instead of hardcoding the URL.
49. Write one more test: test_dropdown_selection(driver) — open the Select Dropdown List demo, use Select(driver.find_element(...)) to choose 'Wednesday', assert the selected option text is 'Wednesday'.

Hint:

- @pytest.mark.parametrize generates N separate test cases — each shows individually in the report as a pass or fail.
- from selenium.webdriver.support.ui import Select is the correct way to interact with <select> HTML elements — avoid clicking options directly.

Expected Outcome: 6 tests run (3 parameterised + checkbox + screenshot hook + dropdown). HTML report shows all results. A failure screenshot is captured if any test fails.

HANDS-ON 7 [Advanced]

Page Object Model (POM) — Design Pattern for Maintainable Tests

Topics Covered:

✓ **POM Design Pattern Principles**	✓ **Creating Page Classes**
✓ **Locator Management in Page Classes**	✓ **Separation of Test Logic from UI Logic**
✓ **Reusability & Maintainability**	✓ **Refactoring Flat Scripts to POM**

The Page Object Model is the single most important design pattern in Selenium automation. It separates 'what to test' (test files) from 'how to interact with the UI' (page files), making suites dramatically easier to maintain when the application changes.

Task 1: Build Page Classes for the Selenium Playground

Goal: Create Page Object classes that encapsulate all locators and actions for each page.

Steps:

50. Create a pages/ folder. Inside, create a BasePage class in base_page.py: it accepts the driver in __init__, and provides common methods: navigate_to(url), get_title(), wait_for_element(locator, timeout=10).
51. Create SimpleFormPage class in simple_form_page.py that inherits BasePage. Define locators as class-level tuples: MESSAGE_INPUT = (By.ID, 'user-message'). Do not hardcode locators inside methods — always reference the class-level tuple.
52. Add interaction methods to SimpleFormPage: enter_message(text) and click_submit() and get_displayed_message(). None of these methods should contain any assert statements — page methods only perform actions and return values.
53. Create CheckboxPage class in checkbox_page.py with methods: check_option(index), uncheck_option(index), is_option_checked(index).
54. Create DropdownPage class in dropdown_page.py with a method select_day(day_name) that uses the Select class internally.

Hint:

- The golden rule of POM: test files contain assertions (what should happen), page files contain interactions (how to make it happen). No driver.find_element calls in test files.
- Locators as class-level constants mean: if the ID changes, you update one line in one file — not every test that uses that element.

Expected Outcome: pages/ folder contains base_page.py, simple_form_page.py, checkbox_page.py, dropdown_page.py. No locators or find_element calls appear in test files.

Task 2: Refactor Tests to Use POM and Build the Full Suite

Goal: Replace all direct driver calls in test files with Page Object method calls.

Steps:

55. Refactor test_simple_form_submission to use SimpleFormPage: page = SimpleFormPage(driver); page.navigate_to(base_url + 'simple-form-demo/'); page.enter_message('Hello Selenium'); page.click_submit(); assert page.get_displayed_message() == 'Hello Selenium'. Verify the test file contains zero driver.find_element calls.
56. Refactor test_checkbox_demo to use CheckboxPage. Refactor test_dropdown_selection to use DropdownPage.
57. Add a new test test_input_form_submit(driver) using a new InputFormPage class. The Input Form Submit page has multiple fields — name, email, phone, address. Create page methods fill_form(name, email, phone, address) and submit_form() and get_success_message(). Assert the form submits successfully.

58. Run the complete test suite: pytest tests/ -v --html=report.html. Verify all tests pass and the report shows the POM-structured test names.
59. In a code comment or README section, explain: what problem would occur in a flat (non-POM) script if the Submit button's ID changed from 'submit' to 'btn-submit'? How does POM solve this?

Hint:

- A fully POM-compliant test file should read like a business requirement, not like HTML interaction code: page.enter_message('Hello') is readable; driver.find_element(By.ID, 'user-message').send_keys('Hello') is not.

Expected Outcome: All tests pass. Zero driver.find_element calls in test files (verify with grep). The maintenance comment correctly explains the POM benefit.